

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Описание входных данных.....	8
1.2 Описание выходных данных.....	8
2 МЕТОД РЕШЕНИЯ.....	10
3 ОПИСАНИЕ АЛГОРИТМОВ.....	13
3.1 Алгоритм конструктора класса Employee.....	13
3.2 Алгоритм метода showName класса Employee.....	13
3.3 Алгоритм метода showRole класса Employee.....	14
3.4 Алгоритм конструктора класса Developer.....	14
3.5 Алгоритм метода showTasks класса Developer.....	14
3.6 Алгоритм метода showRole класса Developer.....	15
3.7 Алгоритм конструктора класса Manager.....	15
3.8 Алгоритм метода showProjects класса Manager.....	15
3.9 Алгоритм метода showRole класса Manager.....	16
3.10 Алгоритм конструктора класса TeamLead.....	16
3.11 Алгоритм метода showLevel класса TeamLead.....	17
3.12 Алгоритм метода showLeadName класса TeamLead.....	17
3.13 Алгоритм метода changeName класса TeamLead.....	17
3.14 Алгоритм метода showRole класса TeamLead.....	18
3.15 Алгоритм функции main.....	18
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	20
5 КОД ПРОГРАММЫ.....	25
5.1 Файл Developer.cpp.....	25
5.2 Файл Developer.h.....	25
5.3 Файл Employee.cpp.....	26
5.4 Файл Employee.h.....	26

5.5 Файл main.cpp.....	27
5.6 Файл Manager.cpp.....	27
5.7 Файл Manager.h.....	28
5.8 Файл TeamLead.cpp.....	28
5.9 Файл TeamLead.h.....	29
6 ТЕСТИРОВАНИЕ.....	30
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	31

1 ПОСТАНОВКА ЗАДАЧИ

В информационной системе технологической компании рассматриваются сотрудники разных ролей. Базовым объектом является сотрудник. На его основе определяются две роли: разработчик и менеджер. Также существует роль тимлида, которая совмещает функции разработчика и менеджера. Для моделирования такой системы необходимо построить ромбовидную иерархию классов.

Базовый класс "Работник"

В защищённом разделе:

- строковое свойство имени сотрудника.

В открытом разделе:

- конструктор с одним строковым параметром;
- метод вывода имени сотрудника в формате "Employee::showName -> <имя>";
- виртуальный метод вывода роли в формате "Employee::showRole -> Employee".

Промежуточный класс "Разработчик"

Класс должен:

- наследоваться от класса Работник так, чтобы избежать ошибки;
- содержать закрытое целочисленное свойство количества выполненных задач;
- иметь конструктор;
- иметь метод вывода количества задач в формате "Developer::showTasks -> <значение>";
- переопределять виртуальный метод вывода роли в формате "Developer::showRole -> Developer".

Промежуточный класс "Менеджер"

Класс должен:

- наследоваться от класса Работник так, чтобы избежать ошибки;
- содержать закрытое целочисленное свойство количества проектов;
- иметь конструктор;
- иметь метод вывода количества проектов в формате

"Manager::showProjects -> <значение>";

- переопределять виртуальный метод вывода роли в формате "Manager::showRole -> Manager".

Производный класс "Тимлид"

Класс должен:

- наследоваться от классов Разработчик и Менеджер;
- содержать закрытое целочисленное свойство уровня руководства;
- иметь конструктор;
- иметь метод вывода уровня руководства в формате

"TeamLead::showLevel -> <значение>";

- иметь метод вывода имени сотрудника из класса Тимлид в формате "TeamLead::showLeadName -> <имя>";

- иметь метод изменения имени сотрудника;
- переопределять виртуальный метод вывода роли в формате

"TeamLead::showRole -> TeamLead".

Алгоритм работы программы

1. Ввести имя сотрудника, количество выполненных задач, количество проектов, уровень руководства.

2. Ввести новое имя.

3. Создать объект производного класса, представляющего тимлида, используя введенные данные.

4. Вывести имя сотрудника через интерфейс базового класса.

5. Вывести имя сотрудника через интерфейс производного класса.
6. Изменить имя сотрудника с использованием функциональности производного класса.
7. Повторно вывести имя сотрудника через интерфейс базового класса.
8. Повторно вывести имя сотрудника через интерфейс производного класса.
9. Вывести количество выполненных задач.
10. Вывести количество проектов.
11. Вывести уровень руководства.
12. Создать три указателя:
 - 12.1. на объект, рассматриваемый как разработчик;
 - 12.2. на объект, рассматриваемый как менеджер;
 - 12.3. на объект, рассматриваемый как сотрудник.
13. Направить все указатели(присвоить адрес) на созданный объект тимлида.
14. Через каждый из указателей выполнить вызов метода вывода роли сотрудника.

1.1 Описание входных данных

Ivan 12 3 2
Petr

1.2 Описание выходных данных

```
Employee::showName -> Ivan  
TeamLead::showLeadName -> Ivan  
Employee::showName -> Petr  
TeamLead::showLeadName -> Petr  
Developer::showTasks -> 12  
Manager::showProjects -> 3  
TeamLead::showLevel -> 2
```

```
TeamLead::showRole -> TeamLead  
TeamLead::showRole -> TeamLead  
TeamLead::showRole -> TeamLead
```

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `lead` класса `TeamLead` предназначен для хранения и управления данными о тимлиде;
- объект `devPtr` класса указатель на объект класса `Developer` предназначен для обращения к объекту тимлида через интерфейс разработчика;
- объект `manPtr` класса указатель на объект класса `Manager` предназначен для обращения к объекту тимлида через интерфейс менеджера;
- объект `empPtr` класса указатель на объект класса `Employee` предназначен для обращения к объекту тимлида через интерфейс сотрудника;
- функция `main` для организации работы программы: ввода исходных данных, создания объекта тимлида, вывода результатов и демонстрации работы виртуальных методов;
- `virtual` - виртуальное наследование.

Класс `Employee`:

- свойства/поля:
 - поле хранение имени сотрудника:
 - наименование — `name`;
 - тип — строка;
 - модификатор доступа — `protected`;
- функционал:
 - метод `Employee` — параметризованный конструктор;
 - метод `showName` — вывод имени сотрудника;

- о метод showRole — виртуальный метод, вывод роли сотрудника (сотрудник).

Класс Developer:

- свойства/поля:
 - о поле хранение количества выполненных задач:
 - наименование — tasks;
 - тип — цел;
 - модификатор доступа — private;
- функционал:
 - о метод Developer — параметризованный конструктор;
 - о метод showTasks — вывод количества выполненных задач;
 - о метод showRole — вывод роли сотрудника (разработчик).

Класс Manager:

- свойства/поля:
 - о поле хранение количества проектов:
 - наименование — projects;
 - тип — цел;
 - модификатор доступа — private;
- функционал:
 - о метод Manager — параметризованный конструктор;
 - о метод showProjects — вывод количества проектов;
 - о метод showRole — вывод роли сотрудника (менеджер).

Класс TeamLead:

- свойства/поля:
 - о поле хранение уровня руководства:
 - наименование — level;
 - тип — цел;

- модификатор доступа — private;
- функционал:
 - о метод TeamLead — параметризированный конструктор;
 - о метод showLevel — вывод уровня руководства;
 - о метод showLeadName — вывод имени сотрудника;
 - о метод changeName — изменение имени сотрудника;
 - о метод showRole — вывод роли сотрудника (тимлид).

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	Employee			Базовый класс сотрудника	
		Developer	virtual public		2
		Manager	virtual public		3
2	Developer			Класс разработчика, виртуально наследуемый от класса сотрудника	
		TeamLead	public		4
3	Manager			Класс менеджера, виртуально наследуемый от класса сотрудника	
		TeamLead	public		4
4	TeamLead			Класса тимлида, наследуемый от классов разработчика и менеджера	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса Employee

Функционал: параметризованный конструктор.

Параметры: строк. name - имя сотрудника.

Алгоритм конструктора представлен в таблице 2.

Таблица 2 – Алгоритм конструктора класса Employee

№	Предикат	Действия	№ перехода
1		присвоение свойству name текущего объекта значение входного параметра name	Ø

3.2 Алгоритм метода showName класса Employee

Функционал: вывод имени сотрудника.

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода showName класса Employee

№	Предикат	Действия	№ перехода
1		вывод на консоль "Employee::showName -> ", name, endl	Ø

3.3 Алгоритм метода showRole класса Employee

Функционал: виртуальный метод, вывод роли сотрудника (сотрудник).

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода showRole класса Employee

№	Предикат	Действия	№ перехода
1		вывод на консоль "Employee::showRole -> Employee" , endl	Ø

3.4 Алгоритм конструктора класса Developer

Функционал: параметризованный конструктор.

Параметры: строк. name - имя сотрудника (передается в качестве аргумента для инициализации объекта сотрудника), цел. tasks - количество выполненных задач.

Алгоритм конструктора представлен в таблице 5.

Таблица 5 – Алгоритм конструктора класса Developer

№	Предикат	Действия	№ перехода
1		присвоение свойству tasks текущего объекта значение входного параметра tasks	Ø

3.5 Алгоритм метода showTasks класса Developer

Функционал: вывод количества выполненных задач.

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода *showTasks* класса *Developer*

№	Предикат	Действия	№ перехода
1		вывод на консоль "Developer::showTasks -> ", tasks, endl	Ø

3.6 Алгоритм метода *showRole* класса *Developer*

Функционал: вывод роли сотрудника (разработчик).

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 7.

Таблица 7 – Алгоритм метода *showRole* класса *Developer*

№	Предикат	Действия	№ перехода
1		вывод на консоль "Developer::showRole -> Developer" , endl	Ø

3.7 Алгоритм конструктора класса *Manager*

Функционал: параметризованный конструктор.

Параметры: строк. name - имя сотрудника (передается в качестве аргумента для инициализации объекта сотрудника), цел. projects - количество проектов.

Алгоритм конструктора представлен в таблице 8.

Таблица 8 – Алгоритм конструктора класса *Manager*

№	Предикат	Действия	№ перехода
1		присвоение свойству projects текущего объекта значение входного параметра projects	Ø

3.8 Алгоритм метода *showProjects* класса *Manager*

Функционал: вывод количества проектов.

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 9.

Таблица 9 – Алгоритм метода *showProjects* класса *Manager*

№	Предикат	Действия	№ перехода
1		вывод на консоль "Manager::showProjects -> ", projects, endl	Ø

3.9 Алгоритм метода *showRole* класса *Manager*

Функционал: вывод роли сотрудника (менеджер).

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 10.

Таблица 10 – Алгоритм метода *showRole* класса *Manager*

№	Предикат	Действия	№ перехода
1		вывод на консоль "Manager::showRole -> Manager" , endl	Ø

3.10 Алгоритм конструктора класса *TeamLead*

Функционал: параметризованный конструктор.

Параметры: строк. name - имя сотрудника, цел. tasks - количество выполненных задач, цел. projects - количество объектов (первые 3 параметра передаются в качестве аргументов для инициализации объектов сотрудника, разработчика и менеджера), цел. level - уровень руководства.

Алгоритм конструктора представлен в таблице 11.

Таблица 11 – Алгоритм конструктора класса TeamLead

№	Предикат	Действия	№ перехода
1		присвоение свойству level текущего объекта значение входного параметра level	Ø

3.11 Алгоритм метода showLevel класса TeamLead

Функционал: вывод уровня руководства.

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 12.

Таблица 12 – Алгоритм метода showLevel класса TeamLead

№	Предикат	Действия	№ перехода
1		вывод на консоль "TeamLead::showLevel -> ", level, endl	Ø

3.12 Алгоритм метода showLeadName класса TeamLead

Функционал: вывод имени сотрудника.

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 13.

Таблица 13 – Алгоритм метода showLeadName класса TeamLead

№	Предикат	Действия	№ перехода
1		вывод на консоль "TeamLead::showLeadName -> ", name, endl	Ø

3.13 Алгоритм метода changeName класса TeamLead

Функционал: изменение имени сотрудника.

Параметры: строк. newName - новое имя сотрудника.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 14.

Таблица 14 – Алгоритм метода *changeName* класса *TeamLead*

№	Предикат	Действия	№ перехода
1		присвоение свойству name значение входного параметра newName	Ø

3.14 Алгоритм метода **showRole** класса **TeamLead**

Функционал: вывод роли сотрудника (тимлид).

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 15.

Таблица 15 – Алгоритм метода *showRole* класса *TeamLead*

№	Предикат	Действия	№ перехода
1		вывод на консоль "TeamLead::showRole -> TeamLead" , endl	Ø

3.15 Алгоритм функции **main**

Функционал: организации работы программы: ввода исходных данных, создания объекта тимлида, вывода результатов и демонстрации работы виртуальных методов.

Параметры: нет.

Возвращаемое значение: цел. код возврата программы.

Алгоритм функции представлен в таблице 16.

Таблица 16 – Алгоритм функции *main*

№	Предикат	Действия	№ перехода
1		объявл. строк. перемен. name и newName	2
2		объявл. цел. перемен. tasks, projects и level	3
3		ввод значений в переменные name, tasks, projects, level	4
4		ввод значения в переменную newName	5
5		создать объект lead класса TeamLead с аргументами name, tasks, projects и level	6
6		вызов метода showName() у объекта lead	7
7		вызов метода showLeadName() у объекта lead	8
8		вызов метода changeName с аргументом newName у объекта lead	9
9		вызов метода showName() у объекта lead	10
10		вызов метода showLeadName() у объекта lead	11
11		вызов метода showTasks() у объекта lead	12
12		вызов метода showProjects() у объекта lead	13
13		вызов метода showLevel() у объекта lead	14
14		иниц. указатель devPtr на объект класса Developer адресом объекта lead	15
15		иниц. указатель manPtr на объект класса Manager адресом объекта lead	16
16		иниц. указатель empPtr на объект класса Employee адресом объекта lead	17
17		вызов метода showRole по указателю devPtr	18
18		вызов метода showRole по указателю manPtr	19
19		вызов метода showRole по указателю empPtr	20
20		возврат 0	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-5.

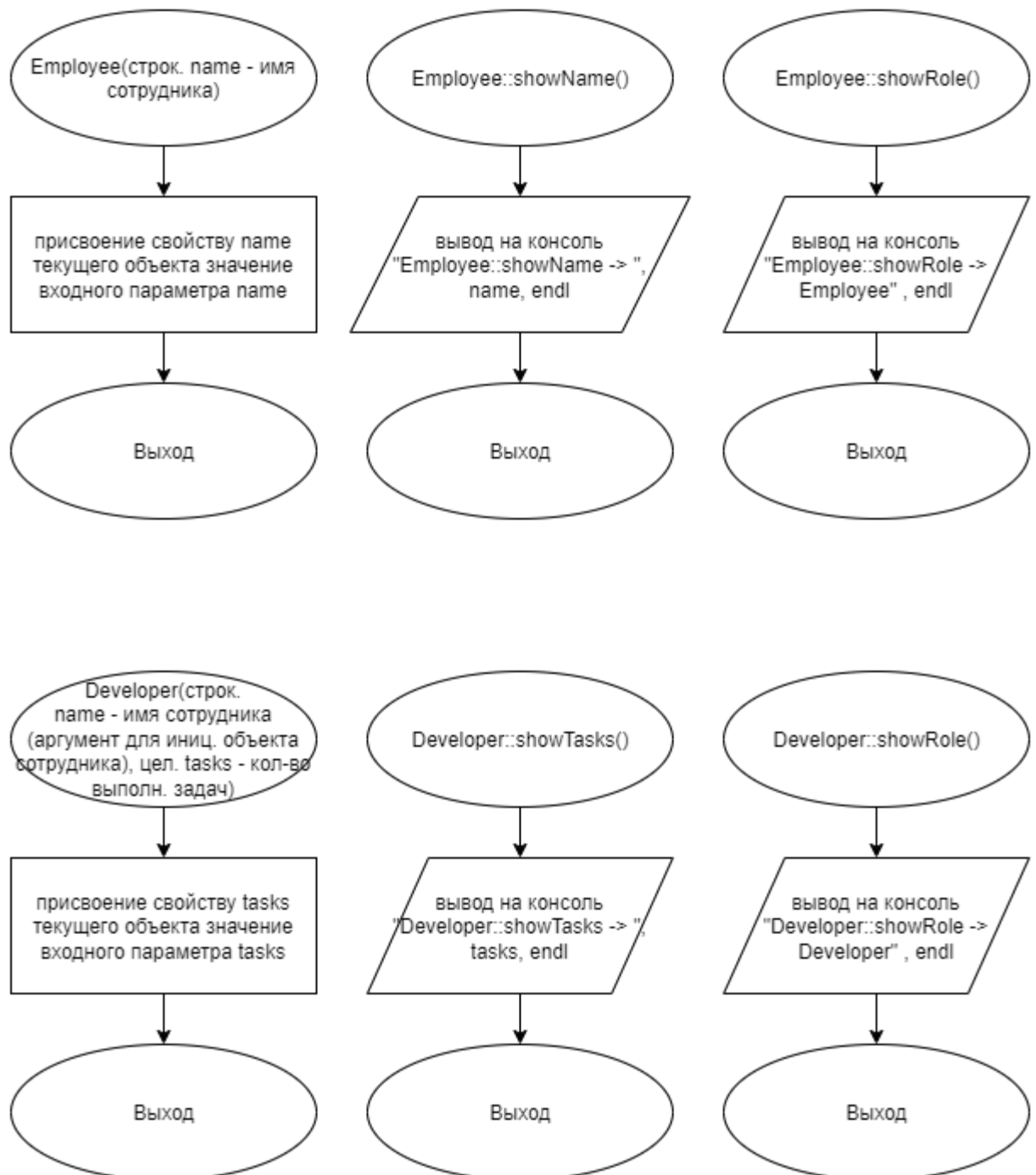


Рисунок 1 – Блок-схема алгоритма

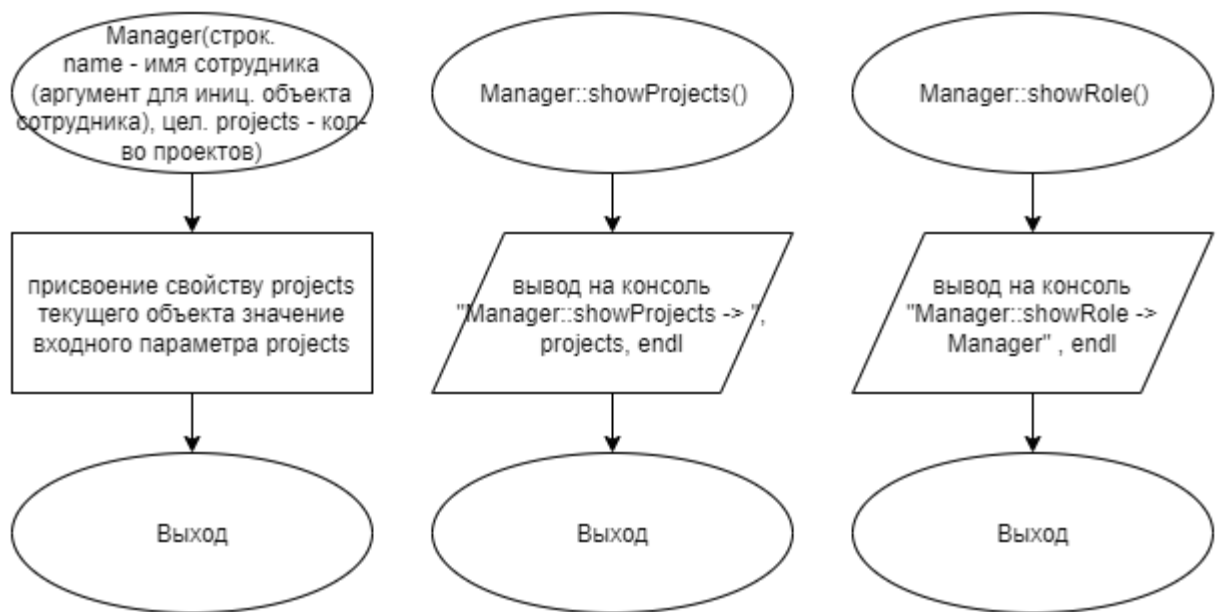


Рисунок 2 – Блок-схема алгоритма

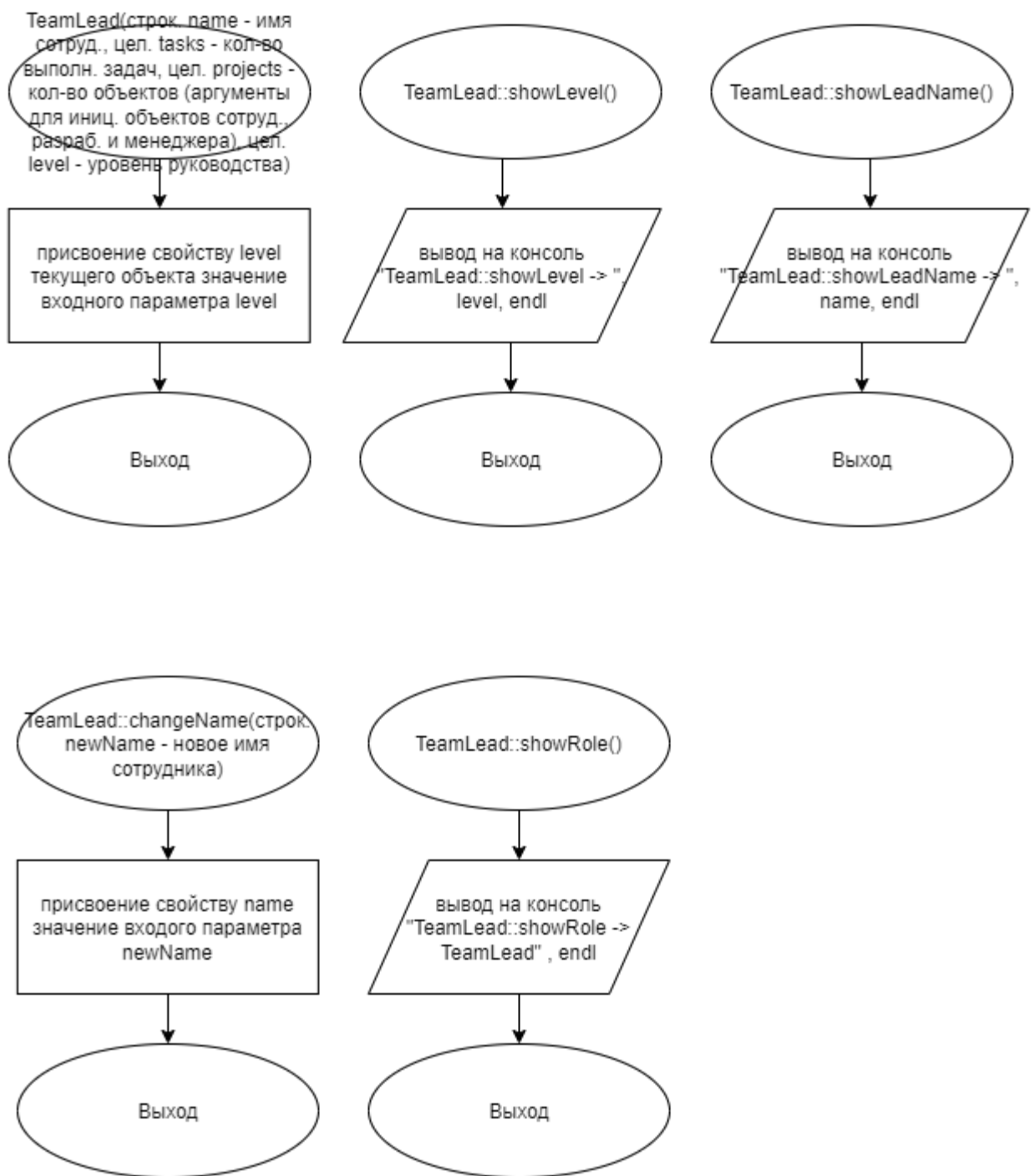


Рисунок 3 – Блок-схема алгоритма

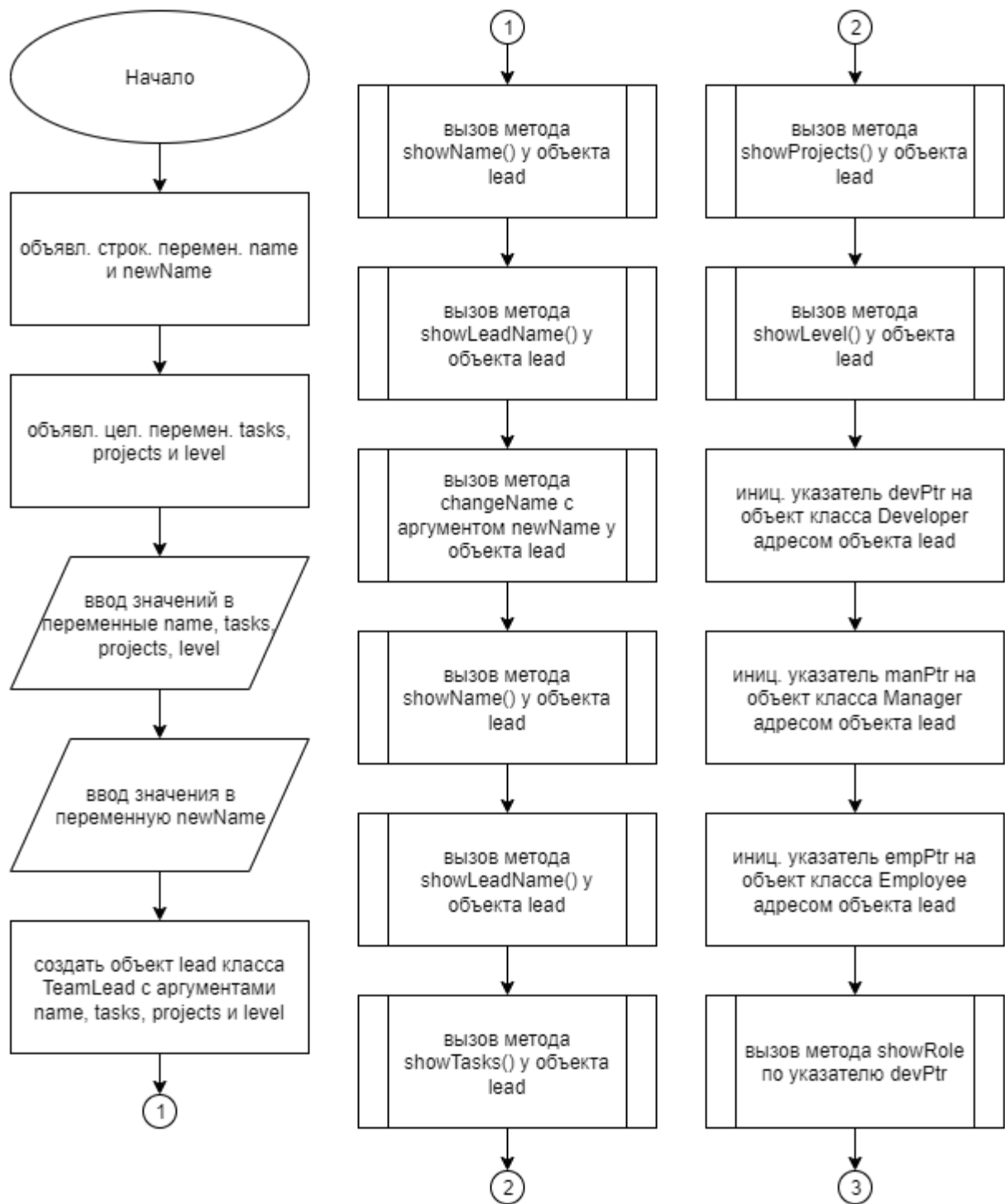


Рисунок 4 – Блок-схема алгоритма

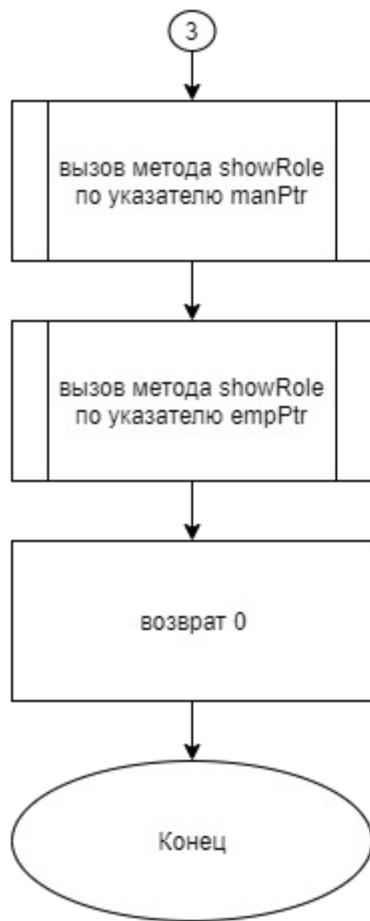


Рисунок 5 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл Developer.cpp

Листинг 1 – Developer.cpp

```
#include "Developer.h"
using namespace std;

Developer::Developer(string name, int tasks) : Employee(name) {
    this->tasks = tasks;
}

void Developer::showTasks() {
    cout << "Developer::showTasks -> " << tasks << endl;
}

void Developer::showRole() {
    cout << "Developer::showRole -> Developer" << endl;
}
```

5.2 Файл Developer.h

Листинг 2 – Developer.h

```
#ifndef __DEVELOPER__H
#define __DEVELOPER__H

#include "Employee.h"

class Developer : virtual public Employee {
private:
    int tasks;

public:
    Developer(string name, int tasks);
    void showTasks();
    void showRole();
};
```

```
#endif
```

5.3 Файл Employee.cpp

Листинг 3 – Employee.cpp

```
#include "Employee.h"
using namespace std;

Employee::Employee(string name) {
    this->name = name;
}

void Employee::showName() {
    cout << "Employee::showName -> " << name << endl;
}

void Employee::showRole() {
    cout << "Employee::showRole -> Employee" << endl;
}
```

5.4 Файл Employee.h

Листинг 4 – Employee.h

```
#ifndef __EMPLOYEE__H
#define __EMPLOYEE__H

#include <string>
#include <iostream>
using namespace std;

class Employee {
protected:
    string name;

public:
    Employee(string name);
    void showName();
    virtual void showRole();
};

#endif
```

5.5 Файл main.cpp

Листинг 5 – main.cpp

```
#include <stdlib.h>
#include <stdio.h>

#include "TeamLead.h"

int main()
{
    string name, newName;
    int tasks, projects, level;

    cin >> name >> tasks >> projects >> level;
    cin >> newName;

    TeamLead lead(name, tasks, projects, level);

    lead.showName();
    lead.showLeadName();

    lead.changeName(newName);

    lead.showName();
    lead.showLeadName();

    lead.showTasks();
    lead.showProjects();
    lead.showLevel();

    Developer* devPtr = &lead;
    Manager* manPtr = &lead;
    Employee* empPtr = &lead;

    devPtr->showRole();
    manPtr->showRole();
    empPtr->showRole();

    return 0;
}
```

5.6 Файл Manager.cpp

Листинг 6 – Manager.cpp

```
#include "Manager.h"
using namespace std;
```

```

Manager::Manager(string name, int projects) : Employee(name) {
    this->projects = projects;
}

void Manager::showProjects() {
    cout << "Manager::showProjects -> " << projects << endl;
}

void Manager::showRole() {
    cout << "Manager::showRole -> Manager" << endl;
}

```

5.7 Файл Manager.h

Листинг 7 – Manager.h

```

#ifndef __MANAGER__H
#define __MANAGER__H

#include "Employee.h"

class Manager : virtual public Employee {
private:
    int projects;

public:
    Manager(string name, int projects);
    void showProjects();
    void showRole();
};

#endif

```

5.8 Файл TeamLead.cpp

Листинг 8 – TeamLead.cpp

```

#include "TeamLead.h"
using namespace std;

TeamLead::TeamLead(string name, int tasks, int projects, int level)
    : Employee(name), Developer(name, tasks), Manager(name, projects) {
    this->level = level;
}

```

```

void TeamLead::showLevel() {
    cout << "TeamLead::showLevel -> " << level << endl;
}

void TeamLead::showLeadName() {
    cout << "TeamLead::showLeadName -> " << name << endl;
}

void TeamLead::changeName(string newName) {
    name = newName;
}

void TeamLead::showRole() {
    cout << "TeamLead::showRole -> TeamLead" << endl;
}

```

5.9 Файл TeamLead.h

Листинг 9 – TeamLead.h

```

#ifndef __TEAMLEAD__H
#define __TEAMLEAD__H

#include "Developer.h"
#include "Manager.h"

class TeamLead : public Developer, public Manager {
private:
    int level;

public:
    TeamLead(string name, int tasks, int projects, int level);
    void showLevel();
    void showLeadName();
    void changeName(string newName);
    void showRole();
};

#endif

```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 17.

Таблица 17 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
Ivan 12 3 2 Petr	Employee::showName - > Ivan TeamLead::showLeadName -> Ivan Employee::showName - > Petr TeamLead::showLeadName -> Petr Developer::showTasks -> 12 Manager::showProjects -> 3 TeamLead::showLevel -> 2 TeamLead::showRole - > TeamLead TeamLead::showRole - > TeamLead TeamLead::showRole - > TeamLead	Employee::showName - > Ivan TeamLead::showLeadName -> Ivan Employee::showName - > Petr TeamLead::showLeadName -> Petr Developer::showTasks -> 12 Manager::showProjects -> 3 TeamLead::showLevel -> 2 TeamLead::showRole - > TeamLead TeamLead::showRole - > TeamLead TeamLead::showRole - > TeamLead

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).